Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

Lab3: Analysis of dynamical systems in biology
Due by Monday April 20th, 2026 @ 11:59pm

The goal of this lab is to learn how to use Matlab to simulate and analyze dynamical systems that use ordinary differential equations. We will numerically solve differential equations using the `ode45` function. We will also address theoretical concepts in dynamical systems such as fixed points and nullclines. We will also perform sensitivity analyses in order to address how dynamics vary with changes to parameter values. Throughout we base our analysis around a model of protein production from mRNA.

1 Simulating coupled ordinary differential equations

According to the central dogma of molecular biology, information encoded in genes within DNA is transcribed into mRNA which is then translated into proteins at the ribosome. Here, we consider the dynamics of a constituitively active promoter, that is, one that constantly recruit RNA polymerase and produces mRNA of the corresponding gene. By assuming an abundance of RNA polymerase, we can assume the mRNA produce protein copies at a constant per capita rate. This leads to dynamics of mRNA concentration, m, and protein concentration, p:

$$\frac{dm}{dt} = a - bm \tag{1}$$

$$\frac{dp}{dt} = rm - bp \tag{2}$$

where a is the rate of mRNA transcription and r is the rate of translation per transcript. The concentration of mRNA and proteins are reduced by dilution at rate b. This dilution occurs because m and p refer to concentrations (number/volume) and the volume of cell grows over time. In total, these equations represent a coupled, ordinary differential equation. These equations happen to be linear, but the framework for analyzing them can apply to both linear and nonlinear equations.

In general, solving equations, like those of mRNA and protein dynamics, can be difficult or impossible in more complex scenarios. However, numerically solving equations like these with a computer is fairly simple in many cases. Here, we will use `ode45` in Matlab, which is run with the following command:

```
[t,x] = ode45(@(t,x) odefun(t,x,pars), [t0 tf],x0);
```

Here, t are points in time, x is the solution of the differential equation, `odefun.m` is the name of the function where you define the differential equation, t0 is the initial time, tf is the final time, x0 is the initial value of x and pars is a structure that can be used to pass parameters into the function `odefun.m`. It is important to note that x can represent a vector of variables, e.g., both m and p. The last two input variables are optional. Hence, if you do not want to pass any variables, then this program can be run as:

```
[t,x] = ode45(@odefun, [t0 tf],x0);
```

The differential equation for the protein model is represented with the function:

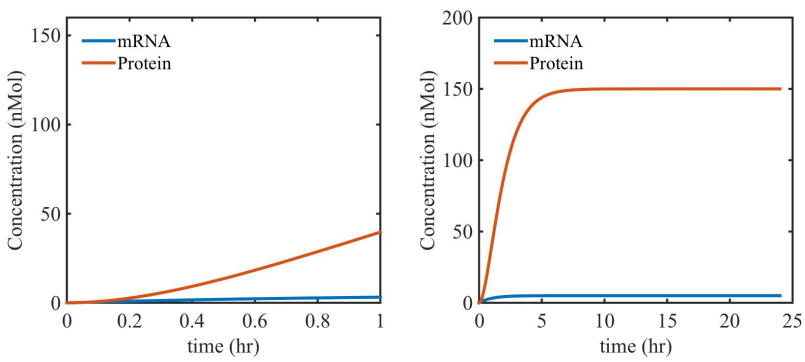
```
function dxdt = proteinproduction(t,x,pars)
m = x(1);
p = x(2);

dmdt = pars.a - pars.b*m;
dpdt = pars.r*m-pars.b*p;

dxdt = [dmdt; dpdt];
end
```

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

In doing so, we have intentionally converted the vector of variables embedded in x into variable names that correspond to the dynamics. This is intentional to help make the code readable, even at the cost of some efficiency. Note, dxdt is a column vector with length equal to the number of variables in the system. The dynamics can be simulated as follows:

```
pars.a = 5; % nM/hr
pars.b = 1; % /hr
pars.r = 30;  % proteins/(mRNA*hr)
x0 = [0 0]; % Initial values
t0 = 0;  % Time to start
tf = 1;  % Final time
[t,x] = ode45(@(t,x) proteinproduction(t,x,pars),[t0 tf],x0);
plot(t,x,'LineWidth',3)
xlabel('time (hr)','FontSize',20)
ylabel('nMol','FontSize',20)
legend('mRNA','Protein')
set(gca,'FontSize',20)
```

The result is plot showing the growth of protein and mRNA over 1 hr. Note the units of time (x-axis) and concentration (y-axis). Next to it is the result of dynamics until the system reaches steady state (as a self-check you should modify the code to simulate out to 24 hrs).



A note on structures: Structures are a convenient way to organize multiple parameters together, both to help you read and re-use code but also to facilitate passing information from scripts to functions. A structure is a set of field-value pairs. The fields have names. In the code above, we have introduced three: a, b, and r. Each has a value, 5, 1, and 30, respectively. However, fields can also denote strings, matrices, and even structures inside structures.

2 Numerically identify nullclines

Here, we identify the nullclines theoretically and plot them with a velocity field to graphically identify the stability of equilibria. A nullcline of a variable is a function in phase space where the respective dynamical equation is set to 0, i.e., corresponding to no rate of change. For the mRNA-protein model, solving for the nullclines leads to:

$$\frac{dm}{dt} = 0 \quad \Longrightarrow \; m = \tfrac{a}{b} \tag{3}$$

$$\frac{dp}{dt} = 0 \quad \Longrightarrow \; m = \tfrac{bp}{r} \tag{4}$$

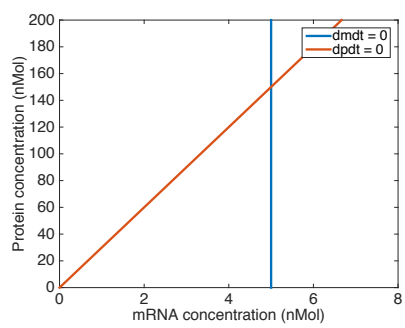
The following commands show how to plot these nullclines in the mRNA-protein space:

```
% Define the nullclines
mnullcline = pars.a/pars.b; % A fixed value
```

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

```
pnullcline = @(p) pars.b.*p./pars.r; % An 'anonymous' function
% Define a vector of protein and mRNA values
pvec = 0:20:200;
mvec = 0:1:8;
% Plot the mRNA nullcline
plot(mnullcline*ones(size(pvec)),pvec,'LineWidth',3)
hold on
% Overlay the protein nullclinej
plot(pnullcline(pvec),pvec,'LineWidth',3)
hold off
% Add some plotting notes and modify the axes
legend('dmdt = 0', 'dpdt = 0')
xlim([min(mvec) max(mvec)])
ylim([min(pvec) max(pvec)])
xlabel('mRNA concentration (nMol)','FontSize',20)
ylabel('Protein concentration (nMol)','FontSize',20)
set(gca,'FontSize',20)
```

The variable `pnullcline` is an anonymous function. The @ symbol means that the input argument 'p' in the assignment is a variable and must be provided as an input when calling pnullcline. Note that like any function, you don't need to call the variable provided to the function by the same name as the local function, and you can even pass in a fixed, scalar value. This code gives:

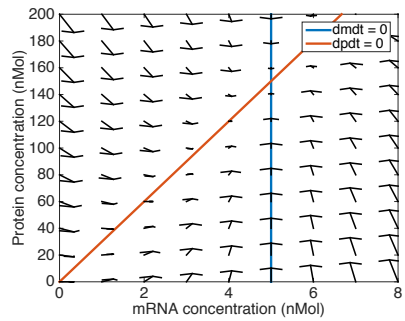


Note that the nullclines cross at precisely one location. At this location both dm/dt = 0 and dp/dt = 0. Hence, the intersection of the lines is the equilibrium of the system. To numerically see the equilibria, we can plot values of $\frac{dm}{dt}$ and $\frac{dp}{dt}$ at points in phase space. One nice way to visualize this is by using arrows that point in the directions given the values. The following code accomplishes this in matlab using the quiver function.

```
dmdt = @(m) pars.a - pars.b.*m;
dpdt = @(m,p) pars.r.*m-pars.b.*p;
[MM,PP] = meshgrid(mvec,pvec);
hold on
quiver(MM,PP,dmdt(MM),dpdt(MM,PP),'k','MaxHeadSize',.015,'LineWidth',2);
hold off
```

Reminder: be careful using the hold command. If the figure gets messy from repeatedly plotting then clear the figure by entering "clf" into the command line. In this code, we made a grid of points based on the vectors mvec and pvec that we defined before. You can choose other points if you prefer.

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.



Of course the nullcline does not necessarily define the actual dynamics. First of all, dynamics can cross nullclines, because they only define where one of the variables (not 2 or all) have zero rate of change. The 'velocity field' depicts the local direction and magnitude of flow.

Exercise 1 (50 pts): Simulate Eqs.(1)-(2) beginning with three initial values in (m, p) space: $(0, 0)$, $(7, 25)$, and $(1, 150)$. What do you expect to happen initially? Will both mRNA and protein increase in all cases? Where should the dynamics converge? Then, overlay the dynamics onto the velocity (vector) field for all three starting conditions, over a 24 hr simulation.

3 Evaluate the local stability of equilibria

Visually at least, the equilibrium appears to be locally stable, in the sense that trajectories approach it asymptotically. In general, there may be multiple locally stable equilibria in a system. Dynamics approach stable equilibria only if they exist within the associated "basin of attraction" (see notes online if you are not familiar with this term). The shape of these basins can be complicated and are not guaranteed to be of a certain size. Hence, to numerically address local stability it is worth first identifying equilibria either analytically by or numerically by solving the intersection of all nullclines. A full accounting of the local stability can usually be assessed through linearization of the dynamical systems near a fixed point. Numerical simulations via small perturbations can be used to verify analytic predictions of local stability or to provide preliminary numerical evidence in the absence of analytical results.

But first, we must identify the equilibrium. As was suggested above, the fixed points of a system $\mathrm{dx_i/dt} = \mathrm{f_i(x}, \theta)$ must satisfy $f_i(x^*, \theta) = 0$ for all x_i. Analytical approaches to solving for equilibria are optimal when possible. Repeatedly solving for variables using the nullclines and plugging into the solutions for the other nullclines solves the algebraic system and yields an equation for equilibria. The present case is straightforward:

$$\frac{dm}{dt} = 0 \qquad \implies m^* = \tfrac{a}{b} \tag{5}$$

$$\frac{dp}{dt} = 0 \quad \implies 0 = rm - bp \implies 0 = \tfrac{ra}{b} - bp \implies p^* = \tfrac{ra}{b^2} \tag{6}$$

The present parameters yield $p^* = 150$ nMol and $m^* = 5$ nMol. Numerical solutions for equilibria can also be used in the event that there are multiple equilibria and solving analytically is difficult, e.g., via the `fzero` command. One approach is to re-arrange the solution of the differential equation at equilibria and provide a guess of the value.

```
allnull = @(p) pars.r.*pars.a/pars.b - pars.b*p;
equil_num = fzero(allnull,100);
```

Here we guess p^* is 100 at equilibria. By varying our guesses throughout parameter space it may be possible to identify potential multiple equilibria. More generally, nullclines are useful in identifying how many equilibria exist in the system.

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

3.1 Jacobians and all that

The local stability of a (nonlinear) dynamical system can be assessed through a series of steps. First, an equilibrium is identified, which we will denote x^*. Second, the nonlinear dynamical systems is linearized, in essence by approximating the full nonlinear dynamics of the system via a Taylor expansion near the fixed point. The full details of this method are described in the lecture notes. This linearized dynamics is described by the Jacobian. Third, the stability of the system is identified by finding the eigenvectors and eigenvalues of the Jacobian. The eigenvectors denote those directions in phase space in which a perturbation will grow or shrink exponentially without changing its direction. In the event that at least one eigenvalue is positive, this implies that small perturbations that have some 'projection' in the associated eigenvector direction will grow exponential. In essence, if one or more eigenvalues has a positive, real component, then the equilibrium, is unstanble. If all eigenvalues have negative real components then the equilibrium is stable. We will return to the borderline case of zero eigenvalues later on in this course.

How then to calculate eigenvalues and eigenvectors? Recall that the Jacobian is the linearized version of dynamics near the fixed point. Let's refer to the small perturbations $u = m - m^*$ and $v = p - p^*$. Becuase the original system was already linear, this transformation yields:

$$\frac{du}{dt} = a - b(u + m^*) \tag{7}$$

$$\frac{dv}{dt} = r(u + m^*) - b(v + p^*) \tag{8}$$

which replacing the fixed point values yields:

$$\frac{du}{dt} = -bu \tag{9}$$

$$\frac{dv}{dt} = ru - bv. \tag{10}$$

This dynamical system is linear – and would be even if the original had been nonlinear if one ignores higher order terms in u and v. As such we expect solutions to be exponential, such that the derivatives with respect to time are just a constant rate factor multiplied by an exponential, i.e., du/dt = λu and dv/dt = λv where λ is the eigenvalue.

We can therefore rewrite these dynamics of perturbations as:

$$\lambda \begin{bmatrix} u \\ v \end{bmatrix} = J \begin{bmatrix} u \\ v \end{bmatrix} \tag{11}$$

where

$$J = \begin{bmatrix} -b & 0 \\ r & -b \end{bmatrix}. \tag{12}$$

The eigenvalues are solution to the equation

$$\mathrm{Det}(J - \lambda I) \tag{13}$$

wehre λ is an eigenvalue and I is the 2×2 identity matrix. Because of the zero in the upper-right of the Jacobian, i.e., the lack of feedback between protein and mRNA, then the two eigvanlues are both $\lambda_{1,2} = -b$. Because both are negative, the system must be locally stable. This result provides the basis for verification in the next section.

3.2 Numerical simulations near equilibria

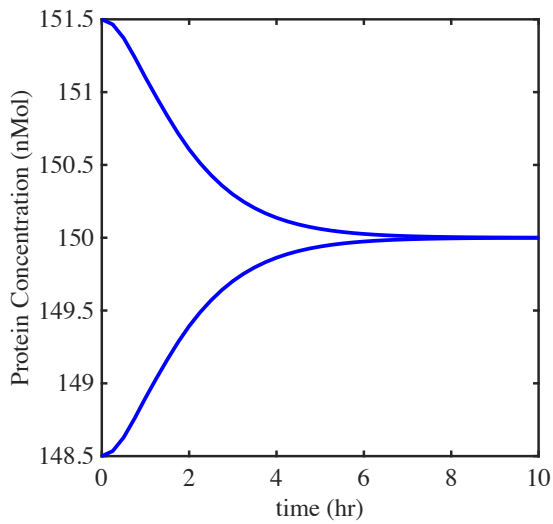
Once we have the equilibria we estimate stability by running the dynamics after perturbing from the equilibria.

```
pstar = pars.r*pars.a/pars.b^2;
mstar = pars.a/pars.b;
```

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

```
pperturb = pstar.*.01;
mperturb = mstar.*.01;

x0 = [mstar+mperturb pstar+pperturb];
t0 = 0;
tf = 10;

[t,x] = ode45(@proteinproduction,[t0 tf],x0,[],pars);
plot(t,x(:,2),'LineWidth',3)
xlabel('time (hr)','FontSize',20)
ylabel('Concentration (nMol)','FontSize',20)
legend('Protein')
set(gca,'FontSize',20)
```

Here we perturbed by 1% from the equilibria. We plot just the protein concentration to make the approach clearer. We would also need to check the mRNA concentration returns to the value as well. **Modify the code so that you can plot two dynamics, one that starts just below and just above the equilibrium, see the result below.**



Exercise 2 (50 pts): Can you characterize the shape of the return of the concentration of proteins to equilibrium with time (0-5 hours for this problem)? Can you predict what it should be from the linearized system of equations and verify the *quantitative* rate of return? Verify this by estimating the fixed point and rate of return for the following parameter sets: $(a = 5, b = 1, r = 30)$, $(a = 5, b = 0.5, r = 30)$ and $(a = 10, b = 1, r = 30)$.

4 Towards bifurcation diagrams – equilibria as parameter values vary

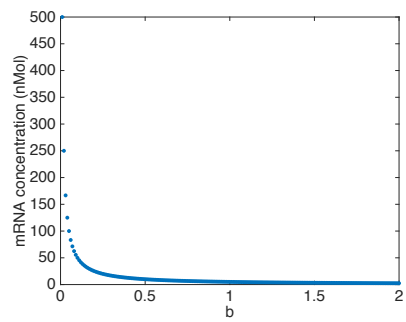
The number of equilibria and possible dynamics in a system can change as the parameter values change (not so much in our example, but in the homework we'll see a richer example). In this example, we will plot the equilibria as we vary the dilution rate, b. To obtain the state diagram:

```
bvec = .01:.01:2;
mstarvec = zeros(1,length(bvec));
```

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

```
pstarvec = zeros(1,length(bvec));
for jj = 1:length(bvec)
   pars.b =  bvec(jj);
   pstarvec(jj) = pars.r*pars.a/pars.b^2;
   mstarvec(jj) = pars.a/pars.b;
end
figure(1)
plot(bvec,mstarvec,'.','MarkerSize',15)
xlabel('b','FontSize',20)
ylabel('mRNA concentration (nMol)','FontSize',20)
set(gca,'FontSize',20)
```

Looking at just the mRNA concentration as we vary b gives the following plot, can you do the same for proteins?

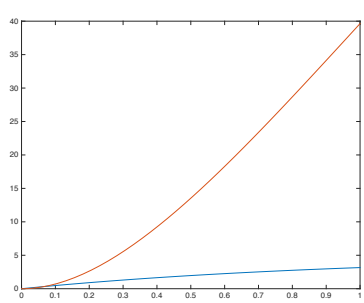


5 A closer look at plotting

Let's consider the dynamics from problem 1, e.g., through the following simple plot command:

```
pars.a = 5;
pars.b = 1;
pars.r = 30;
x0 = [0 0];
t0 = 0;
tf = 1;
[t,x] = ode45(@proteinproduction,[t0 tf],x0,[],pars);
tmph=plot(t,x)
```

which yields:



At first glance, this plot seems difficult to parse. Let's improve things. First, adding a legend allows labeling the lines

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.

```
tmpl=legend('mRNA','Protein');
set(tmpl,'Location','NorthWest');
legend('boxoff');
```

The line names are entered as comma separated strings. Northwest is not the name of a line though. Matlab allows moving the legend by entering a string at the end based on cardinal (North, South, East, West) directions. Similarly, the figure is more informative by labeling the x and y axes

```
xlabel('time (hr)','FontSize',20 )
ylabel('Concentration (nMol)','FontSize',20 )
```

Here, the fontsize string at the end allows changing the size of the font. If we want to do that to the ticklabels and the legend we can change the global font size by setting the current axis

```
set(gca,'FontSize',20)
```

Here "gca" stands for get current axis. Lastly, we can thicken the lines by changing the line width, if we plotted with dots we can set the marker size instead. The following commands may help:

```
set(tmph,'LineWidth',3);
```

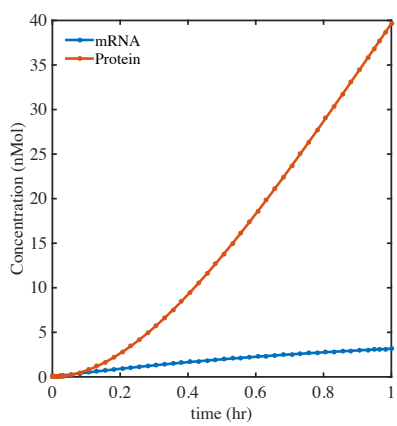
or

```
set(tmph,'Marker','.');
set(tmph,'MarkerSize',20)
```

Keep in mind that we can combine these together:

```
set(tmph,'LineWidth',3,'Marker','.','MarkerSize',20);
```

Now the plot looks like this:



Hurray! This seems nice.

Copyright 2022 - Joshua S. Weitz - Draft version for teaching purposes only. Do not copy, cite, or distribute without permission of the author.